CareConnect — Web Accessibility Documentation

SWEN 661 Team 2 · The Acuity Health Group · Assignment 10, Part 2

SWEN 661 Team 2 (The Acuity Health Group) — Assignment 10, Part 2. The React web app at the repository root (Vite, React Router 7, Tailwind), targeting **WCAG 2.1 Level AA** at the three web breakpoints: mobile 320–767px, tablet 768–1023px, desktop 1024px and up.

This document describes the semantic HTML, ARIA landmarks, skip links, form accessibility, keyboard and focus management, and screen-reader considerations the web app uses, and the component-by-component rules the team follows as the React implementation grows. Every rule is taken from the current source and was checked against the running app (§8); file references let a reviewer trace each claim.

Evidence	Where
Automated audit — 0 violations, 42/42 keyboard & structure checks	`audit/axe-results.png`, `audit/report.html`
Audit before the fixes — 71 violations, 5 failed checks	`audit-before/`
Landmarks, heading outline, links and fields of every page, at every breakpoint	`structure.md`
Accessibility tests (40)	`src/accessibility.test.tsx`
Earlier project-wide notes (WCAG map, memory-loss design)	`ACCESSIBILITY.md`

Who this is for

CareConnect serves **care recipients** living with short-term memory loss (often older, often with low vision or reduced dexterity, sometimes deaf or hard of hearing) and the **caregivers** who manage their medicines and appointments. That sets the bar: every screen must be operable with a keyboard or switch device, must make sense read aloud in order, and must never depend on the user remembering where they were. Most of the rules below follow from those three requirements.

---

1. Semantic HTML structure

Rule: use the HTML element that already means the thing; add ARIA only where HTML cannot say it. A native element brings its role, keyboard behaviour and accessible-name rules with it, in every browser and screen reader.

1.1 Elements and where they are used

Element	Used for	Where
`<header>`	The page banner: logo, role indicator, Switch role / Sign out, and on app pages the orientation bar (date, live time, greeting, current screen, Call my caregiver)	`Layout.tsx`, `Landing.tsx`, `SignIn.tsx`, `SignUp.tsx`, `RoleChooser.tsx`
`<nav>`	Groups of site navigation links only — never a toolbar of actions	App: patient bottom bar / caregiver sidebar (`Layout.tsx`). Landing: Sign in / Sign up (`Landing.tsx`)
`<main id="main-content" tabindex="-1">`	The one primary content area per page; the skip link's and route-change focus target	Every shell
`<footer>`	Site-wide footer: copyright and a "View contacts" help link	`Landing.tsx` (all widths); `Layout.tsx` (desktop only — see §1.3)
`<section aria-label/aria-labelledby>`	A titled part of a page; becomes a named *region*	`Card.tsx` (every card), Today, Appointments date groups, Memories (Pinned / More), dashboard Alerts

Element	Used for	Where
`<article>`	One self-contained item in a feed or list that makes sense on its own: a schedule entry, a medicine, a memory, a contact, a caregiver note	`Schedule.tsx`, `Medications.tsx`, `Memories.tsx`, `Contacts.tsx`, `Caregiver.tsx`
`<aside>`	**Not used.** Nothing in the app is tangential to the main content. The caregiver sidebar is navigation, so it is a `<nav>`, not an `<aside>`. If a "related tips" panel is added, it will be an `<aside aria-label="…">`.	—
`<h1>`–`<h3>`	Exactly one `<h1>` per page, then `<h2>` sections, `<h3>` items, no skipped levels	All pages (outlines in `structure.md`)
`<button>`	Anything that **does** something on this page: toggle, open, delete, switch role, send	Everywhere
`<a href>` / `<Link>`	Anything that **goes** somewhere: another page, a phone number (`tel:`), the skip target	Nav, quick links, Call my caregiver
`<ul role="list">` / `<ol role="list">`	Lists of items; `role="list"` restores list semantics Safari/VoiceOver drops when CSS removes bullets	Nav items, schedule, medicines, appointments
`<form noValidate>`, `<label for>`, `<fieldset>` + `<legend>`	Forms; one visible label per field; groups such as "Schedule times"	`Field.tsx`, Sign in/up, Manage medications/appointments
`<time datetime>`	Dates and times machine-readably	Header clock, schedule and appointment times

Buttons vs links. The test is what happens, not what it looks like: a "Manage medications" card on the dashboard is a link (it goes to a page) even though it looks like a button; "Add to My Day" is a button (it changes this page). Screen readers announce the role, and users choose by it.

1.2 Page skeleton

```
<a class="skip-link" href="#main-content">Skip to main content</a>   ← first Tab stop
<header role="banner">  logo · role · Switch role · Sign out
                        date · time · greeting · current screen · Call my caregiver
</header>
<main id="main-content" tabindex="-1" aria-label="Main content">
  <h1>…</h1>
  <section aria-labelledby="…"><h2 id="…">…</h2> … <article>…</article> </section>
</main>
<nav aria-label="Main navigation"> <ul role="list"> <li><a aria-current="page">…</a> … </ul> </nav>
<footer role="contentinfo"> … </footer>          ← desktop only in the app shell
```

The navigation comes **after** `<main>` in the source on purpose: a keyboard or screen-reader user reaches the page's content first and the menu second. CSS places it visually (bottom bar for the care recipient; left sidebar for the caregiver on desktop via `lg:order-1`) without changing that order.

1.3 How the structure adapts across breakpoints

The landmarks stay the same at every width; only their presentation changes. This matters because a screen-reader user who learned the page on a phone finds the same structure on a desktop.

	Mobile 320–767px	Tablet 768–1023px	Desktop 1024px+
Header	Below 640px Switch role and Sign out are icon-only (named by `aria-label`); date, time and greeting stack	Role pill appears; controls show their text; orientation bar in one row	Same as tablet, wider

	Mobile 320–767px	Tablet 768–1023px	Desktop 1024px+
Patient navigation	Fixed bottom bar, 6 links, icon + label	Fixed bottom bar	Fixed bottom bar
Caregiver navigation	Horizontal scrolling strip under the header	Horizontal strip	Left sidebar with "Switch to care recipient"
Main content	Single column	Single column; short form fields side by side (Manage appointments)	Two-column layouts on Today and Caregiver notes
Footer (`contentinfo`)	Hidden — the bottom bar occupies that edge; "Contacts" is in the nav	Hidden	Shown

Verified per page and breakpoint in `structure.md` ("Shown at" column).

---

2. ARIA landmarks

Landmarks are the table of contents a screen-reader user jumps through (VoiceOver rotor → Landmarks; NVDA `D`; JAWS `R`). CareConnect's rules:

Landmark role	Element	Explicit role kept?	Accessible name	Rule
`banner`	`<header>` (top level)	`role="banner"`	—	Exactly one per page. A `<header>` inside a `<section>` or `<article>` is not a banner, so card headers never add extra ones.
`navigation`	`<nav>`	implicit	"Main navigation" (app), "Account navigation" (landing)	Every `<nav>` is named when a page has, or might have, more than one, so the rotor says which is which.
`main`	`<main>`	implicit	"Main content" / page name ("Sign in", "Choose your role")	Exactly one per page; carries `id="main-content"` and `tabindex="-1"`.
`contentinfo`	`<footer>` (top level)	`role="contentinfo"`	—	At most one per page.
`region`	`<section aria-label>` / `aria-labelledby`	implicit	The section's heading	Only sections with a name become regions; unnamed sections are not landmarks.
`dialog`	`ConfirmDialog`, assistant panel	`role="dialog"`	The dialog's heading via `aria-labelledby`	See §5.

Explicit roles on `<header>` and `<footer>`. `role="banner"` and `role="contentinfo"` repeat what the elements already mean. They are kept on purpose: they are harmless, they make the landmark explicit to anyone reading the source, and they protect against older assistive technology that did not map the HTML5 elements. They are only ever put on the top-level header and footer.

What is not a landmark. Toolbars and button rows inside a page (e.g. the Edit/Delete row on a medicine card) are not wrapped in `<nav>`; they are actions, not navigation.

The audit checks every page at desktop width for one `main`, one top-level `banner` and one `<h1>` (15/15 pass), and axe's landmark rules (`landmark-one-main`, `landmark-no-duplicate-banner`, `landmark-unique`, `region`) report no violations at any breakpoint.

---

3. Skip links

```
<a href="#main-content" className="skip-link">Skip to main content</a>
…
<main id="main-content" tabIndex={-1} …>
```

```
.skip-link        { position: absolute; transform: translateY(-150%); … }   /* off-screen */
.skip-link:focus  { transform: translateY(0); }                             /* slides in  */
```

- **First focusable element on every shell** — landing, sign in, sign up, role chooser ("Skip to role selection"), and the patient and caregiver app shells. Sign in and Sign up had no skip link before this audit; both do now.
- **Visible when focused.** It is moved off-screen, not hidden with `display:none` or `visibility:hidden`, so it stays in the tab order and appears at the top-left in `calm-700` on white (8.7:1) the moment it receives focus.
- **It moves focus, not just the scroll position.** The target `<main>` has `tabindex="-1"`, so following the link puts keyboard focus inside main and the next `Tab` lands on the first control in the content. Without `tabindex="-1"` some browsers scroll but leave focus at the top.
- **Plain anchor, not a router link.** It is an in-page fragment, so React Router does not intercept it and it works before JavaScript has finished loading.
- **No focus ring on `<main>` itself** (`main[tabindex='-1']:focus { outline: none }`): main is a landmark, not a control; the next `Tab` shows the ring on a real control.
- **Focus is never moved on page load.** Two components used to steal it — the assistant pulled focus to its corner button on the landing page, and the confirm dialog focused every page's Delete button — which put the user *past* the skip link. Both are fixed (§5.3); the audit checks it.

Verified by the audit (first `Tab` is the skip link and `Enter` focuses `<main>`, on all four shells) and by tests (`starts with a "Skip to main content" link that targets <main>`, 9 routes).

---

4. Form accessibility

All text inputs go through one component, `Field` (`src/components/Field.tsx`), so the rules below hold on every form; selects use the same pattern (`SelectField` in `ManageAppointments.tsx`).

4.1 Labels

- **Every field has a visible `<label for>`** — never a placeholder-only field. Placeholders are examples ("you@example.com"), not labels, because they vanish as soon as the user types, which is exactly when someone with short-term memory loss needs them.
- **Groups use `<fieldset>` and `<legend>`**: "Schedule times" on the medication form, where one requirement covers several inputs.
- **Icons inside fields are decorative** (`aria-hidden="true"`).

4.2 Required fields

Signal	For whom	Markup
Red ∗ after the label	Sighted users	`<span aria-hidden="true">∗</span>`
"(required)" read with the label	Screen-reader users	`<span class="sr-only">(required)</span>`
Required state	Assistive tech and browser	`required` + `aria-required="true"`

The asterisk is hidden from screen readers so they hear "Email address, required" rather than "Email address star". The caregiver forms used to put "(required)" in the label text *as well*, so it was read and shown twice; that is fixed.

4.3 Error messages

1. The form uses `noValidate` and validates on submit, so error text is CareConnect's own plain-language message, not the browser's tooltip, which disappears and cannot be zoomed or styled.
2. Each field in error gets `aria-invalid="true"`, a 2px `alert-500` border **and** an inline message directly below it, prefixed by an icon — colour is never the only signal.
3. The message is tied to the field with `aria-describedby` (alongside any hint), so a screen reader reads "Email address, invalid, Please enter a valid email address" when focus reaches the field.
4. A summary at the top of the form ("Please correct the errors below to continue.") is `role="alert"`, announced when the form is submitted.
5. Messages say how to fix the problem ("Please enter a valid email address.", "At least one scheduled time is required."), not just that something is wrong.
6. Nothing is cleared on error: entered values stay.

4.4 Autocomplete

Field	`autocomplete`
Your name	`name`
Email address	`email`
Password (sign in)	`current-password`
Password, Confirm password (sign up)	`new-password`

Autofill reduces typing and recall (WCAG 1.3.5), and lets password managers work.

---

5. Keyboard navigation and focus management

5.1 Keyboard model

Every interaction uses native elements, so the keys are the ones users already know:

Key	Does
`Tab` / `Shift+Tab`	Next / previous control, in reading order: skip link → header → page content → navigation → footer
`Enter`	Follow a link; press a button; submit a form from any field
`Space`	Press a button or toggle; tick a checkbox
`Esc`	Close the confirm dialog (focus returns to its trigger); close the assistant (focus returns to its launcher); dismiss the install banner
Arrow keys	Native only: move within a `<select>`, move the caret in a field, scroll the page

There are no single-key shortcuts, so nothing fires when a speech-recognition user dictates text (WCAG 2.1.4).

5.2 Focus visibility and order

- **One focus ring for the whole app**: `*:focus-visible { outline: 3px solid #146488; outline-offset: 3px }` — 6.5:1 on white. It appears for keyboard focus and not for mouse clicks, and is never removed. Verified at every stop by the audit.
- **Targets are at least 44 × 44px** (`min-h/min-w-[2.75rem]`), well above the 24px WCAG 2.2 minimum; bottom-bar links are 56px tall.
- **Focus order follows the source order** (§1.2); CSS `order` is used only for visual placement that keeps content-first reading order.
- **Route changes move focus to** `<main>` (`Layout.tsx`). In a single-page app the document never reloads, so without this a keyboard user who chose "Medicines" in the bottom bar would stay on that link while the content changed silently above it.

Now focus moves to main, the screen reader announces the new page, and the next `Tab` continues from the new content. The window title changes too ("Medicines — CareConnect"). It does not happen on the first load, so the skip link stays the first stop.
- **Current page** is marked with `aria-current="page"` on the nav link, as well as by colour and weight.

5.3 Modal dialogs — focus trap management (`ConfirmDialog.tsx`)

Every destructive action (`<Button variant="danger">`: delete a medication, delete an appointment) asks for confirmation in a modal dialog that follows the WAI-ARIA modal dialog pattern:

Step	Behaviour	Implementation
Name	Announced as "Delete medication, dialog", then its description	`role="dialog"`, `aria-modal="true"`, `aria-labelledby` → the `<h2>`, `aria-describedby` → the paragraph; ids unique per instance
Open	Focus moves into the dialog, to its first control	`querySelectorAll(TABBABLE)[0].focus()` on open
Trap	`Tab` from the last control goes to the first; `Shift+Tab` from the first goes to the last; focus cannot reach the page behind	`keydown` listener on `document` while open
Escape	Closes as Cancel	same listener
Click outside	Closes as Cancel	backdrop `onClick` when the target is the backdrop itself
Close	Focus returns to the button that opened it ("Delete Vitamin D3"), so the user is back where they were	`triggerRef.current.focus()` — **only after the dialog has actually been open**
Background	Page does not scroll behind it; dimming overlay hidden from AT	`body { overflow: hidden }`; overlay `aria-hidden="true"`
Safe default	The safe action (Cancel) is a full button next to the destructive one; the destructive one says exactly what it does ("Yes, delete")	Button labels per call site

The "only after" in the Close row is a fix from this audit: the effect also ran on first render, when the dialog was closed, so loading Manage medications or Manage appointments focused the first **Delete** button — a keyboard user's first `Enter` would have opened a delete confirmation.

The assistant panel is deliberately non-modal (`ChatBot.tsx`): it does not trap focus, because the user may want to read the page while chatting. The launcher exposes `aria-expanded` and `aria-controls`; opening focuses the message input; `Esc` closes it and returns focus to the launcher. Like the dialog, it used to take focus on page load; that is fixed.

5.4 Toggles and grouped controls

- Marking a task or medicine done is a `<button aria-pressed>`, operated with `Space` or `Enter` and reversible at any time. (Its label also changes with the state — see §9.)
- The Memories category filter is a `role="group"` named "Filter memories by category" of `aria-pressed` buttons. The caregiver note priority picker is the same set of `aria-pressed` buttons inside a `<fieldset>` whose `<legend>` is "Priority".

---

6. Screen-reader considerations

6.1 Heading hierarchy

- One `<h1>` per page that says what the page is ("Here's your day, Margaret", "My Appointments", "Dashboard").
- `<h2>` for each section, `<h3>` for items in it; no skipped levels. Headings are chosen for meaning, then styled; a heading is never picked for its size.
- `Card` takes a `headingLevel` prop so the same component nests correctly wherever it is used.
- Fixed in this audit: the appointment date groups ("Today", "Later this week", "Coming up") were styled `<span>`s, so the appointment cards' `<h3>`s sat directly under the `<h1>`. They are now `<h2>`s.

The outline of every page is in `structure.md`; for example:

```
h1  My Appointments                          h1  Dashboard
  h2  Today                                    h2  Margaret's status today
    h3  Blood pressure check — Dr. Sharma      h2  Alerts
  h2  Later this week                          h2  Recent activity
    h3  …                                      h2  Quick links
  h2  Coming up
```

6.2 Meaningful link and button text

- **Links say where they go** out of context, because screen-reader users list links on their own (VoiceOver rotor → Links; NVDA `Insert+F7`): "See full day schedule", "Manage appointments", "View contacts". No "click here" or "read more". The dashboard's "View all" is announced as "View all activity" (visible text plus screen-reader-only text).
- **The accessible name contains the visible words, first** (WCAG 2.5.3 Label in Name), so a voice-control user can say what they see. 18 controls broke this before the audit — for example a link showing "Call my caregiver" was named "Call your caregiver: Joyce…", and "Add to My Day" was named 'Add "Blood test" to My Day schedule'. Now: "Call my caregiver, Joyce — 07700 900 456", "Add to My Day: Blood test". Where the visible text already says everything, the `aria-label` was removed instead.
- **Icon-only controls have a name** ("Close dialog", "Dismiss install prompt", the mobile Switch role / Sign out buttons), and every decorative icon is `aria-hidden="true"`.
- **Never** `aria-label` **on a plain** `<span>` **or** `<div>`: screen readers ignore it. Extra context goes in visually hidden text (`.sr-only`) instead — e.g. "In My Day — already in your My Day schedule".

6.3 Page titles

Every page sets its own `document.title` through `useDocumentTitle` (`src/hooks/useDocumentTitle.ts`): "Medicines — CareConnect", "Sign in — CareConnect", "Dashboard — CareConnect". Before this audit every page shared the title from `index.html`, so the tab, browser history and the screen reader's announcement were the same everywhere.

6.4 Status messages without moving focus

Message	Markup	Announced
Confirmations: checked in, saved, added	`Banner` info/success/warning: `role="status"`, `aria-live="polite"`, `aria-atomic="true"`	After the current sentence
Blocking errors, form summary	`Banner` error, form summary: `role="alert"`	Immediately
"N medicines still to take", schedule all done	`role="status"`	Politely, as the count changes
New caregiver alerts	Region "Alerts", `aria-live="polite"`	Each new alert
Assistant replies, typing indicator	visually hidden `aria-live="polite"` log; `role="status"` "Assistant is typing"	Each reply
Install banner	`role="region"` + `aria-live="polite"`	Once, without taking focus

6.5 Other details

- `<html lang="en">`; dates are formatted with `toLocaleDateString` and also exposed as `<time datetime>`.
- Lists keep list semantics after the CSS reset (`role="list"`), so VoiceOver says "list, 6 items".
- Colour is never the only signal: statuses carry an icon **and** a word ("Done", "Now", "Coming up"); errors carry an icon, a border and text.
- Text contrast is at least 4.5:1 everywhere (see §8: 45 elements failed before).

---

7. Component reference — semantics, keyboard and ARIA

The implementation plan the team follows when building or changing a component.

Component	Element(s)	Keyboard	Name / ARIA	Notes
Skip link	`<a href="#main-content">`	Tab (first), Enter	Text "Skip to main content"	Off-screen until focused
App header (`Layout`)	`<header role="banner">`, `<time>`	Tab through controls	Icon-only controls have `aria-label`s that start with any visible text	Sticky
Main navigation	`<nav aria-label="Main navigation">` → `<ul role="list">` → `NavLink`	Tab, Enter	`aria-current="page"` on the current link	After `<main>` in source
Route change	`<main tabindex="-1">`	—	Focus moves to main; title updates	Not on first load
Button (`Button.tsx`)	`<button>`	Enter, Space	Visible text; `aria-label` only when icon-only	`danger` variant always opens `ConfirmDialog`
Confirm dialog	`role="dialog" aria-modal="true"`	Focus in, Tab trapped, Esc, return focus	`aria-labelledby`, `aria-describedby`	§5.3
Assistant	`<button aria-expanded aria-controls>` + `role="dialog"` panel	Enter opens, Esc closes, Tab can leave	Panel labelled by its heading; live log	Non-modal
Text field (`Field.tsx`)	`<label for>` + `<input>`/`<textarea>`	Native	`aria-required`, `aria-invalid`, `aria-describedby` (hint, error)	§4
Select (`SelectField`)	`<label for>` + `<select>`	Arrows, type-ahead	Same as Field	Native list, no custom combobox
Field group	`<fieldset>` + `<legend>`	Native	Legend names the group	Schedule times
Done toggle	`<button aria-pressed>`	Space, Enter	Fixed name ("Breakfast done") + `aria-pressed` — target pattern, see §9	Reversible
Filter group	`role="group"` `aria-label` + `aria-pressed` buttons	Tab, Space	Group name	Memories, note priority
Card (`Card.tsx`)	`<section aria-label>` + `<hN>`	—	Region named by its heading	`headingLevel` prop
List items	`<article>` inside `<li>`	—	`aria-label` on the article where the visible text is split	Schedule, medicines, memories, contacts
Banner (`Banner.tsx`)	`<div role="status	alert">`	Dismiss button if dismissible	`aria-live`, `aria-atomic`
Progress (My Day)	`role="progressbar"`	—	`aria-valuenow/min/max`, `aria-label`	Also shown as text
Install prompt	`role="region" aria-labelledby` `aria-live="polite"`	Tab to Install / Not now / Dismiss; Esc	Dismiss has `aria-label`	Never takes focus
Tooltips (planned, Part 1 design system)	Trigger `<button aria-describedby>` + `role="tooltip"`	Show on focus and hover; Esc hides	Never holds essential information	WCAG 1.4.13

Component	Element(s)	Keyboard	Name / ARIA	Notes
Tabs (planned)	`role="tablist/tab/tabpanel"`	Arrows move, Home/End, Tab into panel	`aria-selected`, `aria-controls`	Use links instead if each tab is a page
Dropdown menu (planned)	`<button aria-expanded aria-haspopup="menu">` + `role="menu"`	Enter/Space/↓ opens, arrows, Esc returns focus		Prefer a native `<select>` for choosing a value
Hamburger menu (planned, mobile web)	`<button aria-expanded aria-controls>` "Menu" + the same `<nav>`	Enter/Space toggles, Esc closes and returns focus	Name "Menu"; `aria-expanded`	Not a modal unless it covers the page; if it does, trap focus as §5.3

---

8. Verification and results

Automated audit — `npm run a11y:web` (`scripts/web-a11y-audit.mjs`): axe-core 4.14 with the WCAG 2.0/2.1 A and AA rule sets, best practices and the label-in-name rule, in Chromium, over 15 routes and 3 interactive states (sign-in errors, the assistant open, the confirm dialog open) at 375px, 820px and 1280px (54 scans), plus 42 keyboard and structure checks.

	Before	After
axe violations (54 scans)	**71**	**0**
— colour contrast (elements)	153	0
— label in name (elements)	75	0
— heading order (elements)	3	0
Keyboard & structure checks	36 / 41	**42 / 42**
Accessibility tests	0	**40** (296 in the suite, all passing)

Defects found and fixed

#	Defect	WCAG	Fix
1	Body text in `neutral-400` (2.85:1) and other sub-4.5:1 shades across 17 files; done items faded with `opacity-60`; the "not done" circle at 1.65:1	1.4.3, 1.4.11	Text uses the next darker shade; opacity removed; circle 5.5:1. Palette comments corrected to measured ratios — several claimed 4.5:1+ for shades that were 2.9–4.3:1
2	Confirm dialog focused every page's Delete button on load	2.4.3	Return focus only after a real close
3	Assistant pulled focus to its corner button on load, ahead of the skip link	2.4.1, 2.4.3	Same fix
4	No skip link on Sign in or Sign up	2.4.1	Added; `<main tabindex="-1">`
5	One document title for every page	2.4.2	`useDocumentTitle` on every page
6	Caregiver's home announced as "Current screen: Home"	2.4.6	"Dashboard"
7	Focus stayed on the nav link after navigating	2.4.3, 4.1.3	Focus moves to `<main>` on route change
8	18 controls whose name did not contain their visible text	2.5.3	Names start with the visible words, or `aria-label` removed
9	`aria-label` on a plain `<span>` (ignored by AT)	4.1.2	Moved to `.sr-only` text
10	Appointment cards (`h3`) directly under the `h1`	1.3.1	Date groups are `<h2>`
11	"(required)" read and shown twice on caregiver forms	3.3.2	Removed from label text

#	Defect	WCAG	Fix
12	"View all" link ambiguous out of context	2.4.4	"View all activity" for screen readers

9. Known gaps and next steps

Gap	Plan
The page behind the modal dialog is not `inert`; `aria-modal` relies on the screen reader honouring it	Set `inert` on the app root while a dialog is open
Each field error is also `role="alert"`, so submitting a form with four errors announces five times	Keep the summary as the only alert; field errors are read through `aria-describedby` when focused; move focus to the first invalid field
Done toggles (My Day, Medicines) change their label with the state ("Mark Breakfast as done" ↔ "…as not done") **and** set `aria-pressed`, so a screen reader hears "Mark Breakfast as not done, toggle button, pressed" — two state signals that read as contradictory	Keep `aria-pressed` and give the button a fixed name ("Breakfast done"), as the WAI-ARIA toggle-button pattern advises
The app footer (`contentinfo`) is hidden below 1024px	Acceptable for now (its one link, Contacts, is in the nav); revisit with the Part 1 responsive designs
No manual screen-reader pass on the web app yet	VoiceOver (Safari, macOS and iOS) and NVDA (Chrome/Firefox, Windows) through the main flows before the next submission
Tooltips, tabs, dropdowns and the mobile hamburger menu are designed (Part 1) but not built	Build to the patterns in §7, with a test each

How to run the checks

```
npm install
npx playwright install chromium          # once
npm run dev -- --port 5180               # terminal 1
npm run a11y:web                         # terminal 2 — axe + keyboard checks → docs/web/accessibility/audit/
npm run a11y:structure                   # landmarks, headings, links → structure.md
npm test                                 # includes src/accessibility.test.tsx
```